

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHÂN HIỆU TẠI TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ



BÁO CÁO ĐỒ ÁN MÔN HỌC
NGÔN NGỮ LẬP TRÌNH CHO IOT
ĐỀ TÀI: HỆ THỐNG CHẤM CÔNG QUỆT THẺ RFID ĐỒNG BỘ
VỚI GOOGLE SHEET

Giảng viên hướng dẫn: ThS. Võ Thiện Linh

Sinh viên thực hiện:

Cao Xuân Phụng - 6351020056 - CQ.63.KS.ĐTTH&CN

TP. Hồ Chí Minh - Năm 2026

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHÂN HIỆU TẠI TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ



BÁO CÁO ĐỒ ÁN MÔN HỌC
NGÔN NGỮ LẬP TRÌNH CHO IOT
ĐỀ TÀI: HỆ THỐNG CHẤM CÔNG QUỆT THẺ RFID ĐỒNG BỘ
VỚI GOOGLE SHEET

Giảng viên hướng dẫn: ThS. Võ Thiện Linh

Sinh viên thực hiện:

Cao Xuân Phụng - 6351020056 - CQ.63.KS.ĐTTH&CN

TP. Hồ Chí Minh - Năm 2026

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

Giảng viên hướng dẫn

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến ThS. Võ Thiện Linh. Trong suốt quá trình học tập và tìm hiểu môn học Ngôn ngữ lập trình cho IoT, thầy đã luôn tận tâm giảng dạy, truyền đạt cho em những kiến thức nền tảng quý báu. Đặc biệt, sự hướng dẫn sát sao và những góp ý chuyên môn từ thầy chính là yếu tố then chốt giúp em hoàn thành đồ án này.

Đề tài "Hệ thống chấm công quét thẻ RFID đồng bộ với Google Sheet sử dụng ESP32" là một thử thách thú vị, đòi hỏi sự kết hợp chặt chẽ giữa việc xử lý phần cứng vi điều khiển và kỹ năng lập trình giao thức mạng. Nhờ có sự định hướng phương pháp tư duy từ thầy, em đã từng bước gỡ rối các vấn đề kỹ thuật và hiện thực hóa thành công hệ thống.

Mặc dù đã nỗ lực hết sức trong việc nghiên cứu và triển khai dự án, nhưng do giới hạn về mặt thời gian cũng như kinh nghiệm thực tiễn, cuốn báo cáo đồ án này chắc chắn vẫn còn những thiếu sót nhất định. Em rất mong nhận được sự thông cảm, những lời nhận xét và góp ý quý báu từ thầy để đề tài có thể hoàn thiện hơn, đồng thời giúp em rút ra được những bài học kinh nghiệm cho các dự án sau này.

Kính chúc thầy luôn dồi dào sức khỏe, hạnh phúc và gặt hái được nhiều thành công hơn nữa trong sự nghiệp "trồng người".

Trân trọng,

Sinh viên thực hiện

MỤC LỤC

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN	1
LỜI CẢM ƠN.....	2
MỤC LỤC.....	3
DANH MỤC HÌNH ẢNH.....	5
DANH MỤC TỪ VIẾT TẮT	6
CHƯƠNG 1: TỔNG QUAN	8
1.1. Giới thiệu đề tài:.....	8
1.2. Mục tiêu của đề tài:.....	8
1.3. Phạm vi ứng dụng:.....	9
CHƯƠNG 2: GIỚI THIỆU PHẦN MỀM THỰC HIỆN VÀ LINH KIỆN	9
2.1. Phần mềm thực hiện:	9
2.1.1: Arduino IDE:	9
2.1.2: Google Sheet:.....	11
2.2. Linh kiện:.....	13
2.2.1. ESP 32:.....	13
2.2.2. Module RFID RC522:.....	15
2.2.3. LCD 1602 tích hợp module chuyển đổi I2C LCD 8050 R7:	17
2.2.4. Còi Buzzer:	19
CHƯƠNG 3: THÍ NGHIỆM VÀ KẾT QUẢ THỰC HIỆN.....	21
3.1. Tạo bảng thông tin trên Google Sheet:	21
3.2. Sản phẩm hoàn thiện:	24
3.2.1. Sơ đồ kết nối chân:.....	24
3.2.2. Sơ đồ Schematic:	24
3.2.3. Sơ đồ mạch in 3D:	25
3.2.4. Sản phẩm thực tế sau hoàn thiện:	25
3.2.5. Source Code:.....	27
CHƯƠNG 4: KẾT LUẬN	30
4.1. Kết quả đạt được:	30

4.1.1. Về mặt lý thuyết:	30
4.1.2. Về mặt thực nghiệm:.....	30
4.2. Hạn chế còn tồn đọng:	31
4.3. Hướng phát triển trong tương lai:.....	32
TÀI LIỆU THAM KHẢO	32

DANH MỤC HÌNH ẢNH

Hình 2.1: Biểu tượng của Arduino IDE	10
Hình 2.2: Biểu tượng của Google Sheet.....	11
Hình 2.3: Các chân của ESP 32	13
Hình 2.4: Module RFID RC522	16
Hình 2.5: LCD tích hợp module chuyển đổi I2C LCD 9050 R7	18
Hình 2.6: Còi Buzzer	20
Hình 3.1: Tiện ích Apps Script trên Google Sheet.....	21
Hình 3.2: Giao diện Apps Scripts trên Google Sheet.....	22
Hình 3.3: Sơ đồ kết nối chân hoàn thiện	24
Hình 3.4: Sơ đồ Schematic	24
Hình 3.5: Sơ đồ mạch in 3D	25
Hình 3.6: Mặt trên sản phẩm thực tế sau hoàn thiện	26
Hình 3. 7: Bên trong hộp của sản phẩm thực tế sau hoàn thiện	26

DANH MỤC TỪ VIẾT TẮT

ADC: Analog-to-Digital Converter - Bộ chuyển đổi tín hiệu tương tự sang số

BLE: Bluetooth Low Energy - Công nghệ Bluetooth năng lượng thấp

DAC: Digital-to-Analog Converter - Bộ chuyển đổi tín hiệu số sang tương tự

EEPROM: Electrically Erasable Programmable Read-Only Memory - Bộ nhớ chỉ đọc có thể xóa và lập trình bằng điện

GAS: Google Apps Script - Ngôn ngữ kịch bản đám mây của Google

GPIO: General-Purpose Input/Output - Chân tín hiệu đầu vào/đầu ra đa dụng

HTTP/HTTPS: Hypertext Transfer Protocol / Secure - Giao thức truyền tải siêu văn bản / Phiên bản bảo mật

I2C (I²C): Inter-Integrated Circuit - Chuẩn giao tiếp dữ liệu nối tiếp đồng bộ

IDE: Integrated Development Environment - Môi trường phát triển tích hợp

IoT: Internet of Things - Internet vạn vật

LCD: Liquid Crystal Display - Màn hình hiển thị tinh thể lỏng

MAC: Media Access Control - Điều khiển truy nhập môi trường

OTP: One-Time Programmable - Bộ nhớ chỉ lập trình một lần

PCB: Printed Circuit Board - Bảng mạch in

PWM: Pulse Width Modulation - Phương pháp điều chế độ rộng xung

RFID: Radio Frequency Identification - Công nghệ nhận dạng đối tượng qua sóng vô tuyến

ROM: Read-Only Memory - Bộ nhớ chỉ đọc

RTC: Real-Time Clock - Bộ đếm thời gian thực

SCL: Serial Clock - Dây tín hiệu xung nhịp (trong giao tiếp I2C)

SDA: Serial Data - Dây tín hiệu truyền dữ liệu (trong giao tiếp I2C)

SoC: System on Chip - Hệ thống được tích hợp trên một vi mạch

SPI: Serial Peripheral Interface - Chuẩn giao tiếp ngoại vi nối tiếp

UART: Universal Asynchronous Receiver-Transmitter - Bộ truyền nhận dữ liệu nối tiếp không đồng bộ

UID: Unique Identifier - Mã định danh duy nhất (của thẻ RFID)

WPA/WPA2: Wi-Fi Protected Access - Giao thức bảo mật mạng không dây

CHƯƠNG 1: TỔNG QUAN

1.1. Giới thiệu đề tài:

Trong bối cảnh công nghệ thông tin phát triển mạnh mẽ như hiện nay, quy mô nhân sự tại các tổ chức, doanh nghiệp ngày càng mở rộng. Điều này đặt ra một bài toán khó cho công tác quản lý nhân sự, bởi việc kiểm soát số lượng lớn lao động bằng các phương pháp truyền thống đã dần bộc lộ nhiều hạn chế và kém hiệu quả. Để giải quyết vấn đề này, việc ứng dụng hệ thống chấm công tự động là một giải pháp thiết yếu. Bằng cách cấp cho mỗi cá nhân một thẻ định danh (RFID) để điểm danh điện tử, hệ thống không chỉ giúp tối ưu hóa quy trình quản lý mà còn tiết kiệm đáng kể thời gian giám sát ngày công. Trong đề tài này, hệ thống thu thập dữ liệu chấm công được vận hành bởi vi điều khiển ESP32. Ngay khi nhân viên thực hiện thao tác quẹt thẻ, dữ liệu điểm danh sẽ được xử lý và đồng bộ trực tiếp lên nền tảng trực tuyến Google Sheets theo thời gian thực, mang lại sự tiện lợi, chính xác và minh bạch trong công tác lưu trữ.

1.2. Mục tiêu của đề tài:

Mục tiêu tổng quát của đề tài là nghiên cứu, thiết kế và xây dựng một hệ thống chấm công tự động, ứng dụng công nghệ nhận dạng qua sóng vô tuyến (RFID) kết hợp vi điều khiển ESP32 nhằm số hóa và tối ưu hóa quy trình quản lý nhân sự.

Để đạt được mục tiêu trên, đề tài tập trung hoàn thành các mục tiêu cụ thể sau:

- Về mặt thiết kế phần cứng: Tích hợp thành công đầu đọc RFID với vi điều khiển trung tâm ESP32, đảm bảo thiết bị hoạt động ổn định, có khả năng đọc và giải mã chính xác mã định danh (UID) từ thẻ vật lý của người dùng.
- Về mặt kết nối và truyền thông: Khai thác tối đa thế mạnh kết nối Wi-Fi của ESP32 để thiết lập đường truyền dữ liệu không dây ổn định, hỗ trợ gửi thông tin log chấm công từ thiết bị lên môi trường Internet.
- Về mặt quản lý dữ liệu đám mây: Xây dựng cơ chế tương tác và đồng bộ dữ liệu tự động, giúp nền tảng Google Sheets tiếp nhận thông tin kịp thời và lưu trữ chính xác vào các hàng (row) trong bảng tính theo thời gian thực.

- Về mặt ứng dụng thực tiễn: Cung cấp cho nhà quản lý một công cụ giám sát trực quan, hỗ trợ việc theo dõi, trích xuất và phân tích dữ liệu lịch sử điểm danh của nhân viên, từ đó tối giản hóa công tác tính toán thời gian làm việc.

1.3. Phạm vi ứng dụng:

Hệ thống chấm công bằng thẻ RFID kết hợp nền tảng Google Sheets được thiết kế với khả năng ứng dụng linh hoạt trong nhiều môi trường khác nhau như: các cơ sở giáo dục, doanh nghiệp, tổ chức và cơ quan hành chính. Giải pháp này đáp ứng hiệu quả nhu cầu giám sát thời gian làm việc và học tập của nhiều đối tượng, từ sinh viên đến cán bộ, nhân viên. Việc đưa hệ thống vào thực tiễn không chỉ giúp tự động hóa hoàn toàn quy trình điểm danh truyền thống mà còn nâng cao tính minh bạch, khách quan và đảm bảo độ chính xác tuyệt đối trong công tác ghi nhận ngày công.

CHƯƠNG 2: GIỚI THIỆU PHẦN MỀM THỰC HIỆN VÀ LINH KIẾN

2.1. Phần mềm thực hiện:

2.1.1: Arduino IDE:

Tổng quan về Arduino IDE Arduino IDE (Integrated Development Environment) là một môi trường phát triển tích hợp mã nguồn mở, được thiết kế bởi Arduino Software. Đây là công cụ phần mềm đóng vai trò trung gian, cho phép người dùng soạn thảo mã nguồn (viết code), biên dịch (compile) và nạp (upload) chương trình điều khiển vào bộ nhớ của các dòng vi điều khiển. Arduino IDE sử dụng ngôn ngữ lập trình chủ đạo là C và C++ đã được tinh chỉnh, đi kèm với các hàm API thân thiện, giúp đơn giản hóa quá trình lập trình phần cứng mà không đòi hỏi người dùng phải can thiệp quá sâu vào các tập lệnh phức tạp của hệ thống.



Hình 2.1: Biểu tượng của Arduino IDE

Các thành phần và tính năng chính Phần mềm cung cấp một giao diện người dùng trực quan với các công cụ cốt lõi hỗ trợ đặc lực cho quá trình phát triển hệ thống IoT:

- Trình soạn thảo mã (Text Editor): Khu vực viết mã nguồn với các tính năng đánh dấu cú pháp, tự động căn lề và kiểm tra lỗi cơ bản.
- Trình biên dịch (Compiler): Chuyển đổi mã nguồn C/C++ thành ngôn ngữ máy (mã hex) mà vi điều khiển có thể hiểu và thực thi.
- Trình quản lý thư viện (Library Manager): Cung cấp một hệ sinh thái mã nguồn mở khổng lồ. Người dùng có thể dễ dàng tìm kiếm, cài đặt và tích hợp các thư viện hỗ trợ giao tiếp với các module ngoại vi.
- Màn hình theo dõi cổng nối tiếp (Serial Monitor): Công cụ dùng để giao tiếp hai chiều giữa máy tính và vi điều khiển. Tính năng này đặc biệt quan trọng trong việc gỡ lỗi (debug), cho phép hiển thị các thông số trạng thái hệ thống, log dữ liệu hoặc thông tin định danh thẻ đọc được theo thời gian thực.

Sự phù hợp với hệ thống phát triển Mặc dù được phát triển ban đầu dành riêng cho các bo mạch thuộc họ Arduino, Arduino IDE sở hữu tính linh hoạt cao nhờ công cụ Board

Manager. Tính năng này cho phép tích hợp và lập trình cho các dòng vi điều khiển của bên thứ ba, điển hình là lõi vi xử lý ESP32.

Đối với hệ thống thiết kế trong đồ án, Arduino IDE là lựa chọn tối ưu nhờ cộng đồng hỗ trợ mạnh mẽ và hệ thống thư viện đa dạng. Phần mềm cung cấp đầy đủ các thư viện chuyên dụng để ESP32 có thể xử lý giao thức SPI khi giao tiếp với đầu đọc thẻ RFID, đồng thời quản lý hiệu quả các ngăn xếp mạng (network stack) để kết nối Wi-Fi, xử lý giao thức HTTPS và thực hiện các lệnh POST/GET nhằm đồng bộ dữ liệu điểm danh lên nền tảng Google Sheets một cách ổn định và bảo mật.

2.1.2: Google Sheet:

Tổng quan về Google Sheets Google Sheets là một ứng dụng bảng tính trực tuyến, hoạt động dựa trên nền tảng điện toán đám mây (cloud-based), nằm trong hệ sinh thái dịch vụ Google Workspace do Google cung cấp. Không chỉ đơn thuần là một công cụ soạn thảo và xử lý số liệu tương tự như các phần mềm bảng tính truyền thống, Google Sheets cho phép người dùng khởi tạo, chỉnh sửa, định dạng và chia sẻ dữ liệu hoàn toàn trên môi trường Internet. Mọi thay đổi đều được lưu trữ tự động và đồng bộ hóa theo thời gian thực.



Hình 2.2: Biểu tượng của Google Sheet

Các tính năng cốt lõi hỗ trợ quản lý

- Lưu trữ và truy xuất đa nền tảng: Dữ liệu được lưu trữ an toàn trên máy chủ của Google, cho phép người quản lý truy cập, theo dõi và xuất báo cáo từ bất kỳ thiết bị nào có kết nối Internet (máy tính, điện thoại di động) mà không bị phụ thuộc vào không gian và thời gian.
- Xử lý và trực quan hóa dữ liệu: Cung cấp đầy đủ các hàm tính toán logic, công cụ lọc, sắp xếp và vẽ biểu đồ, giúp tự động hóa quá trình tổng hợp ngày công và xuất báo cáo thống kê một cách trực quan.
- Tính năng chia sẻ và phân quyền: Hỗ trợ tính năng cấp quyền truy cập linh hoạt (chỉ xem, nhận xét hoặc chỉnh sửa), đảm bảo tính bảo mật của cơ sở dữ liệu chấm công đối với từng cấp bậc nhân sự trong tổ chức.

Vai trò và cơ chế tích hợp trong hệ thống IoT Trong khuôn khổ của đề tài, Google Sheets không chỉ đóng vai trò là giao diện hiển thị mà còn hoạt động như một hệ quản trị cơ sở dữ liệu (Database) trực tuyến để lưu trữ log sự kiện điểm danh. Mấu chốt để hệ thống phần cứng (ESP32) có thể giao tiếp được với nền tảng này là nhờ vào Google Apps Script (GAS) – một ngôn ngữ kịch bản đám mây dựa trên JavaScript.

Thay vì phải thuê và xây dựng các máy chủ (Server) độc lập tốn kém, Google Apps Script cho phép tạo ra một ứng dụng web (Web App) đóng vai trò như một API trung gian. Khi có người dùng quét thẻ, vi điều khiển ESP32 sẽ đóng gói dữ liệu định danh (UID) và gửi một yêu cầu HTTP/HTTPS (phương thức GET hoặc POST) có chứa dữ liệu này đến địa chỉ Web App. Mã kịch bản (Script) tại đây sẽ tiếp nhận yêu cầu, trích xuất thông tin, tự động đính kèm thời gian thực (Time-stamp) và ghi trực tiếp vào một hàng (row) mới trên bảng tính Google Sheets.

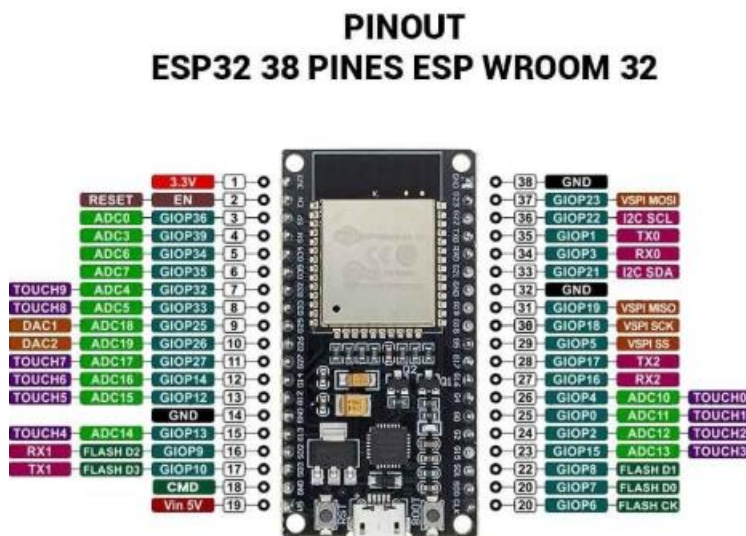
2.2. Linh kiện:

2.2.1. ESP 32:

Tổng quan về vi điều khiển ESP32 là dòng vi điều khiển tiên tiến, chi phí thấp và tối ưu hóa năng lượng, nổi bật với khả năng tích hợp đồng thời kết nối Wi-Fi và Bluetooth.

Được phát triển bởi Espressif Systems dựa trên quy trình công nghệ 40 nm, ESP32 là thế hệ kế nhiệm và khắc phục nhiều hạn chế của dòng vi điều khiển ESP8266 trước đó, mang lại độ linh hoạt cao cho các thiết bị không dây và ứng dụng Internet vạn vật (IoT).

Kiến trúc bên trong của ESP32 sử dụng vi xử lý Tensilica Xtensa LX6 (hỗ trợ tùy chọn lõi đơn hoặc lõi kép). SoC này được tích hợp sẵn các thành phần vô tuyến quan trọng (RF) như ăng-ten, bộ khuếch đại công suất, bộ khuếch đại thu nhiễu thấp (LNA) và module quản lý năng lượng, giúp thu gọn tối đa kích thước mạch phát triển.



Hình 2.3: Các chân của ESP 32

Thông số kỹ thuật chi tiết

1. Khả năng kết nối không dây:

- Wi-Fi: Hỗ trợ chuẩn giao tiếp IEEE 802.11 b/g/n.
- Bluetooth: Hỗ trợ Bluetooth v4.2 BR/EDR và Bluetooth Low Energy (BLE), sử dụng cơ chế chia sẻ sóng vô tuyến với Wi-Fi.

2. Giao tiếp ngoại vi (Peripheral Interfaces): Thiết bị cung cấp 34 chân GPIO vật lý đa dụng, có thể cấu hình linh hoạt cho nhiều chuẩn giao tiếp và tính năng khác nhau:

- Xử lý tín hiệu Analog: 18 kênh ADC SAR 12-bit, 2 bộ DAC 8-bit, 10 chân cảm ứng điện dung (Touch sensor) và bộ tiền khuếch đại analog công suất cực thấp (Ultra-low power analog pre-amplifier).
- Giao tiếp đồng bộ/bất đồng bộ: * 3 bộ SPI (SPI, HSPI và VSPI) hoạt động ở cả 2 chế độ Master/Slave. Trong đó SPI0 và SPI1 dùng để kết nối bộ nhớ flash nội bộ, SPI2 (HSPI) và SPI3 (VSPI) dành cho ngoại vi.
 - 2 bộ I²C (Master/Slave) hỗ trợ Standard mode (100 Kbit/s) và Fast mode (400 Kbit/s), cùng 2 chế độ định địa chỉ 7-bit/10-bit.
 - 2 bộ I²S phục vụ xử lý tín hiệu âm thanh.
 - 3 bộ UART (UART0, UART1, UART2) với tốc độ truyền tải lên đến 5 Mbps.
- Ngoại vi chuyên dụng khác: Bộ điều khiển SD/SDIO/CE-ATA/MMC/eMMC (Host/Slave); Giao diện Ethernet MAC hỗ trợ DMA và chuẩn thời gian chính xác IEEE 1588; CAN bus 2.0; Bộ điều khiển hồng ngoại từ xa (TX/RX lên đến 8 kênh); PWM hỗ trợ điều khiển động cơ và LED (lên đến 16 kênh); Tích hợp sẵn cảm biến hiệu ứng Hall.

3. Cơ chế bảo mật (Security):

- Khả năng mã hóa mạng không dây đáp ứng các chuẩn IEEE 802.11 bao gồm WFA, WPA/WPA2 và WAPI.
- Hỗ trợ các cơ chế bảo mật cấp phần cứng: Secure boot (Khởi động an toàn) và Mã hóa bộ nhớ Flash.
- Tích hợp vùng nhớ OTP 1024-bit (dành tối đa 768-bit cho người dùng).
- Tích hợp bộ tăng tốc mã hóa phần cứng mạnh mẽ: AES, SHA-2, RSA, Mật mã đường cong Ellip (ECC) và Trình tạo số ngẫu nhiên (RNG).

4. Quản lý năng lượng (Power Management):

- Tích hợp bộ ổn áp nội với điện áp rơi thấp (Internal low-dropout regulator).
- Cung cấp miền nguồn hoạt động độc lập (Individual power domain) cho bộ đếm thời gian thực (RTC).
- Hỗ trợ 5 chế độ tiêu thụ năng lượng từ cao đến thấp: *Active*, *Modem-sleep*, *Light-sleep*, *Deep-sleep* và *Hibernation*, đáp ứng các yêu cầu khắt khe về pin của thiết bị IoT.
- Khả năng đánh thức hệ thống (Wake-up) linh hoạt thông qua ngắt GPIO, Timer, quá trình đo ADC hoặc từ ngắt cảm ứng điện dung.

2.2.2. Module RFID RC522:

Tổng quan về thiết bị Module RFID RC522 là một thiết bị nhận dạng đối tượng qua sóng vô tuyến (Radio Frequency Identification) được thiết kế dựa trên lõi vi mạch MFRC522 của hãng NXP Semiconductors. Nhờ ưu điểm về độ chính xác, kích thước nhỏ gọn và chi phí thấp, công nghệ này hiện đang được ứng dụng rộng rãi trong các hệ thống kiểm soát truy cập, điểm danh điện tử, nhận dạng cá nhân và quản lý theo dõi vật phẩm trong thực tế.



Hình 2.4: Module RFID RC522

Đặc điểm và Thông số kỹ thuật

- Tần số hoạt động: Module hoạt động ổn định ở dải tần số vô tuyến 13.56 MHz. Đây là băng tần tiêu chuẩn và được sử dụng phổ biến nhất trên toàn cầu cho các ứng dụng RFID tầm gần (NFC/RFID).
- Chuẩn thẻ hỗ trợ: Thiết bị có khả năng tương tác linh hoạt với các loại thẻ (Tag) RFID chuẩn 13.56 MHz. Điển hình là khả năng đọc và ghi đối với họ thẻ MIFARE (như MIFARE Classic 1K, MIFARE Classic 4K, MIFARE Ultralight) cùng các dòng thẻ có chuẩn giao tiếp tương thích.
- Giao thức kết nối (Giao tiếp SPI): RFID RC522 sử dụng chuẩn giao tiếp đồng bộ tốc độ cao SPI (Serial Peripheral Interface) làm phương thức kết nối chủ đạo với vi điều khiển trung tâm (như ESP32). Giao thức này cho phép quá trình truyền và nhận dữ liệu diễn ra liên tục, nhanh chóng và dễ dàng cấu hình về mặt phần cứng.
- Khả năng thao tác dữ liệu: Không chỉ giới hạn ở việc quét mã định danh (UID), chip MFRC522 còn hỗ trợ khả năng thao tác dữ liệu hai chiều. Người dùng có thể lập trình để đọc thông tin từ các khối nhớ (Block) trên thẻ hoặc ghi đè dữ liệu mới lên thẻ, phục vụ linh hoạt cho các bài toán quản lý phức tạp.

- Khoảng cách giao tiếp (Read Range): Phạm vi quét thẻ hiệu dụng của module phụ thuộc phần lớn vào thiết kế ăng-ten tích hợp trên board mạch và đặc tính cấu tạo của bản thân thẻ RFID. Trong điều kiện tiêu chuẩn, module đảm bảo khả năng đọc/ghi ổn định ở khoảng cách từ 2 cm đến 10 cm.
- Hỗ trợ phần mềm đa nền tảng: Nhờ tính phổ biến cao, module RC522 được cộng đồng công nghệ hỗ trợ hệ sinh thái mã nguồn mở rất mạnh mẽ. Thiết bị đi kèm với nhiều bộ thư viện hoàn chỉnh, tương thích với đa dạng ngôn ngữ lập trình và môi trường phát triển như Arduino IDE (cho ESP32, Arduino), Python (cho Raspberry Pi) và các nền tảng nhúng khác.

2.2.3. LCD 1602 tích hợp module chuyển đổi I2C LCD 8050 R7:

Tổng quan về thiết bị Trong các hệ thống nhúng, việc cung cấp giao diện phản hồi trực quan cho người dùng là vô cùng cần thiết. Màn hình tinh thể lỏng LCD 1602 (Liquid Crystal Display) là một thiết bị hiển thị tiêu chuẩn, được ứng dụng rộng rãi nhờ tính ổn định và tiêu thụ ít điện năng.

Điểm nổi bật của cấu hình phần cứng trong hệ thống này là việc màn hình LCD được tích hợp sẵn một mạch chuyển đổi giao tiếp I2C (mang mã định danh LCD 8050 R7). Sự kết hợp này giúp khắc phục hoàn toàn nhược điểm tốn quá nhiều chân tín hiệu của chuẩn giao tiếp song song (Parallel) truyền thống, mang lại giải pháp kết nối tối ưu cho vi điều khiển ESP32.



Hình 2.5: LCD tích hợp module chuyển đổi I2C LCD 9050 R7

Đặc điểm và Thông số kỹ thuật

- Khả năng hiển thị và kích thước vật lý: Màn hình có kích thước tổng thể 84x44mm. Đúng như tên gọi "1602", thiết bị có độ phân giải hiển thị gồm 16 cột và 2 hàng, cho phép hiển thị tối đa 32 ký tự alpha-numeric cùng lúc. Màn hình đi kèm đèn nền (Backlight) giúp dễ dàng quan sát trong các điều kiện thiếu sáng.
- Giao thức kết nối I2C (Inter-Integrated Circuit): Module chuyển đổi đóng vai trò là cầu nối trung gian, mã hóa dữ liệu thành chuẩn I2C. Nhờ đó, việc giao tiếp giữa ESP32 và màn hình LCD được tối giản xuống chỉ còn 4 chân: VCC (cấp nguồn), GND (nối đất), và 2 dây tín hiệu là SDA (Serial Data - truyền dữ liệu) và SCL (Serial Clock - cấp xung nhịp). Điều này giúp tiết kiệm tối đa các chân GPIO quý giá trên ESP32 cho các ngoại vi khác.

- Cơ chế điều khiển tối ưu: Thay vì phải xuất tín hiệu điều khiển trên 6 đến 8 chân độc lập, người lập trình chỉ cần gửi các tập lệnh chuẩn qua bus I2C. Trong quá trình lập trình, việc xác định đúng địa chỉ phần cứng (Slave Address) của module I2C là yếu tố bắt buộc để ESP32 có thể "nhận diện" và giao tiếp thành công với màn hình (các địa chỉ phổ biến thường là 0x27, 0x3F hoặc có thể thay đổi tùy thuộc vào cấu hình chân trên module).
- Hỗ trợ tập ký tự đa dạng: Bo mạch điều khiển bên trong LCD 1602 cung cấp một bộ ROM chứa các ký tự chuẩn. Thiết bị hỗ trợ hiển thị tốt bảng chữ cái tiếng Anh, chữ số, các ký tự đặc biệt (dấu câu, ký hiệu toán học). Ngoài ra, bộ nhớ CGRAM (Character Generator RAM) còn cho phép người dùng tự thiết kế và định nghĩa các ký tự tùy chỉnh (Custom characters) để tạo ra các biểu tượng đồ họa riêng biệt phục vụ cho ứng dụng.

2.2.4. Còi Buzzer:

Tổng quan về thiết bị Còi chip (Buzzer) là một linh kiện điện tử cơ bản có chức năng chuyển đổi tín hiệu điện thành tín hiệu âm thanh. Trong các hệ thống nhúng và thiết bị IoT, còi Buzzer thường được tích hợp để đóng vai trò như một kênh giao tiếp phi ngôn ngữ (audio feedback), giúp hệ thống cung cấp các phản hồi trạng thái hoạt động một cách trực quan và tức thời đến người sử dụng.



Hình 2.6: Còi Buzzer

Đặc điểm kỹ thuật và phân loại

- Phân loại: Buzzer thường được chia thành hai dòng cơ bản: còi chủ động (Active Buzzer) và còi thụ động (Passive Buzzer). Trong các ứng dụng chỉ yêu cầu âm báo đơn giản, còi chủ động là lựa chọn tối ưu. Thiết bị này đã được tích hợp sẵn mạch dao động bên trong, do đó chỉ cần cấp nguồn điện một chiều (DC) là có thể phát ra âm thanh ở một tần số cố định (thường là tiếng "bíp").
- Điện áp và kết nối: Thiết bị hoạt động ổn định ở dải điện áp thấp (phổ biến từ 3.3V đến 5V), hoàn toàn tương thích với mức điện áp logic của vi điều khiển ESP32.
- Cơ chế điều khiển: Việc vận hành còi chủ động rất đơn giản và tiết kiệm tài nguyên. Người lập trình chỉ cần sử dụng một chân tín hiệu Digital (GPIO) bất kỳ trên ESP32, xuất tín hiệu logic mức cao (HIGH) hoặc mức thấp (LOW) để bật/tắt thiết bị.

Vai trò của Buzzer trong hệ thống chấm công Mặc dù là một linh kiện phụ trợ nhỏ, Buzzer lại đóng vai trò rất quan trọng trong việc nâng cao trải nghiệm người dùng (UX) và tính chuyên nghiệp của hệ thống điểm danh:

- Xác nhận thao tác hợp lệ: Khi nhân viên đưa thẻ vào vùng nhận diện của module RFID và hệ thống đọc mã UID thành công, ESP32 sẽ điều khiển Buzzer phát ra một tiếng "bíp" ngắn. Phản hồi này giúp người dùng an tâm rằng thao tác vật lý của họ đã được thiết bị ghi nhận, ngay cả khi họ không quan sát màn hình LCD.
- Cảnh báo trạng thái lỗi: Trong các trường hợp phát sinh sự cố như: thẻ không hợp lệ (chưa được đăng ký trong hệ thống), thiết bị mất kết nối Wi-Fi cục bộ, hoặc lỗi gián đoạn khi đồng bộ dữ liệu lên máy chủ Google Sheets, hệ thống có thể được lập trình để phát ra các chuỗi âm thanh cảnh báo ngắt quãng (ví dụ: 3 tiếng "bíp" liên tục). Chức năng này giúp phát hiện và xử lý lỗi kịp thời tại điểm danh.

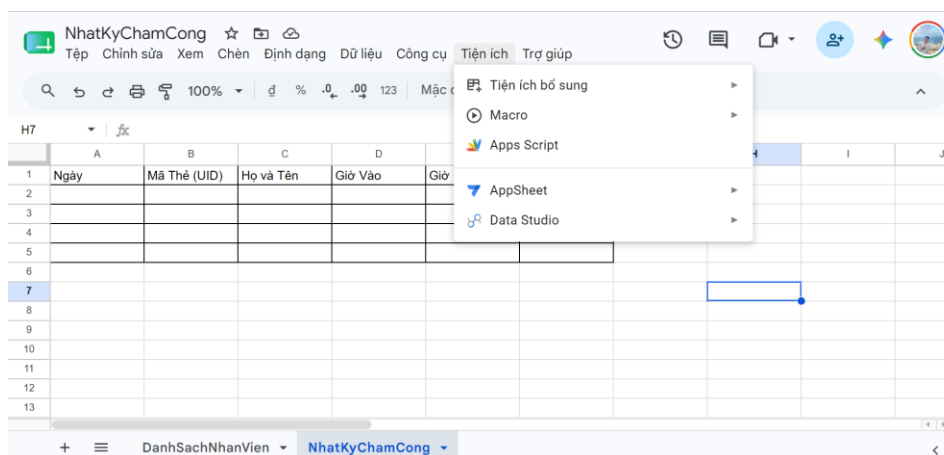
CHƯƠNG 3: THÍ NGHIỆM VÀ KẾT QUẢ THỰC HIỆN

3.1. Tạo bảng thông tin trên Google Sheet:

Các bước để thêm 1 Script vào Google Sheet:

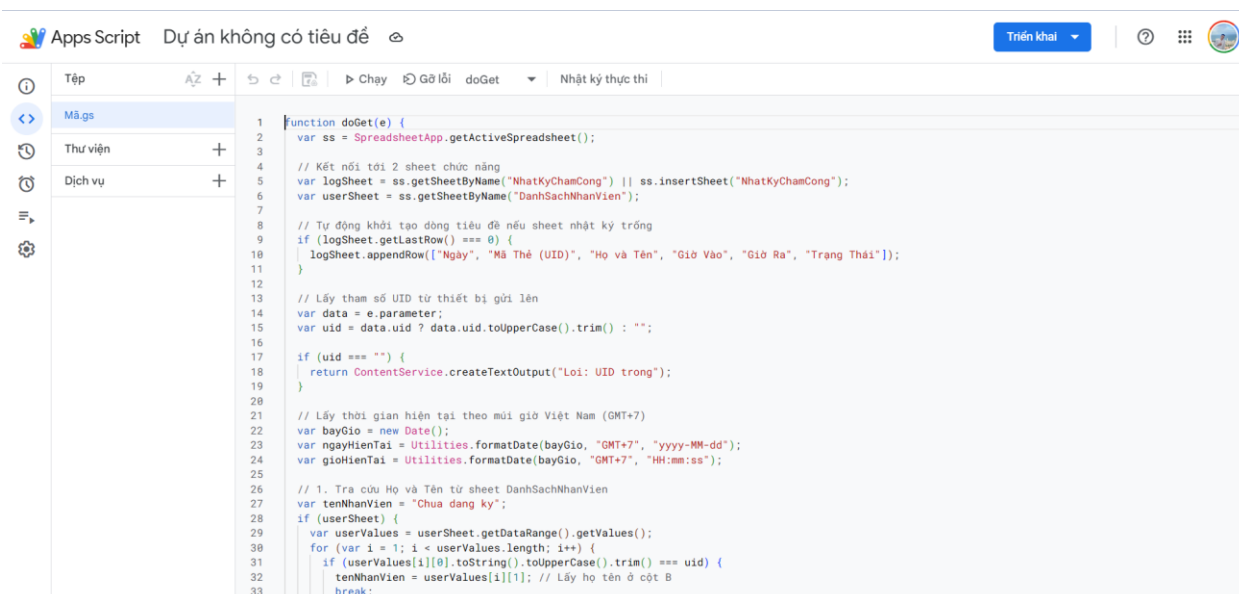
Bước 1: Mở cửa sổ trang tính

Bước 2: Vào Tiện ích mở rộng -> Apps Script



Hình 3.1: Tiện ích Apps Script trên Google Sheet

Bước 3: Trong Script Editor, điền mã lệnh



Hình 3.2: Giao diện Apps Scripts trên Google Sheet

Bước 4: Lưu dự án và chạy chương trình

```
function doGet(e) {
  var ss = SpreadsheetApp.getActiveSpreadsheet();

  // Kết nối tới 2 sheet chức năng
  var logSheet = ss.getSheetByName("NhatKyChamCong") ||
ss.insertSheet("NhatKyChamCong");
  var userSheet = ss.getSheetByName("DanhSachNhanVien");

  // Tự động khởi tạo dòng tiêu đề nếu sheet nhật ký trống
  if (logSheet.getLastRow() === 0) {
    logSheet.appendRow(["Ngày", "Mã Thẻ (UID)", "Họ và Tên", "Giờ Vào", "Giờ Ra",
"Trạng Thái"]);
  }

  // Lấy tham số UID từ thiết bị gửi lên
  var data = e.parameter;
  var uid = data.uid ? data.uid.toUpperCase().trim() : "";

  if (uid === "") {
    return ContentService.createTextOutput("Lỗi: UID trong");
  }

  // Lấy thời gian hiện tại theo múi giờ Việt Nam (GMT+7)
  var bayGio = new Date();
  var ngayHienTai = Utilities.formatDate(bayGio, "GMT+7", "yyyy-MM-dd");
```



```

var gioHienTai = Utilities.formatDate(bayGio, "GMT+7", "HH:mm:ss");

// 1. Tra cứu Họ và Tên từ sheet DanhSachNhanVien
var tenNhanVien = "Chua dang ky";
if (userSheet) {
    var userValues = userSheet.getDataRange().getValues();
    for (var i = 1; i < userValues.length; i++) {
        if (userValues[i][0].toString().toUpperCase().trim() === uid) {
            tenNhanVien = userValues[i][1]; // Lấy họ tên ở cột B
            break;
        }
    }
}

// 2. Kiểm tra nhật ký xem nhân viên này đã quét thẻ hôm nay chưa
var logValues = logSheet.getDataRange().getValues();
var dongDaCo = -1;

for (var j = 1; j < logValues.length; j++) {
    var ngayTrongSheet = Utilities.formatDate(new Date(logValues[j][0]), "GMT+7",
"yyyy-MM-dd");
    var uidTrongSheet = logValues[j][1].toString().toUpperCase().trim();

    // Nếu trùng ngày và trùng mã thẻ tức là đã có lịch sử trong ngày
    if (ngayTrongSheet === ngayHienTai && uidTrongSheet === uid) {
        dongDaCo = j + 1; // Lưu lại vị trí dòng dữ liệu
        break;
    }
}

// 3. Thực hiện logic ghi nhận Check-in / Check-out
if (dongDaCo === -1) {
    // Lướt quét đầu tiên trong ngày -> Ghi nhận Giờ Vào (Check-in)
    logSheet.appendRow([ngayHienTai, uid, tenNhanVien, gioHienTai, "", "Check-in
OK"]);
    return ContentService.createTextOutput("Vao: " + tenNhanVien);
} else {
    // Đã có giờ vào hôm nay -> Tiến hành kiểm tra để ghi nhận Giờ Ra (Check-out)
    var gioRaHienTai = logValues[dongDaCo - 1][4];

    if (gioRaHienTai === "") {
        // Nếu cột Giờ Ra trống -> Cập nhật giờ ra vào cột E (Cột số 5)
        logSheet.getRange(dongDaCo, 5).setValue(gioHienTai);
        logSheet.getRange(dongDaCo, 6).setValue("Hoan thanh");
        return ContentService.createTextOutput("Ra: " + tenNhanVien);
    } else {
        // Nếu đã quét đầy đủ cả 2 lần trước đó
        return ContentService.createTextOutput("Da du cong: " + tenNhanVien);
    }
}

```

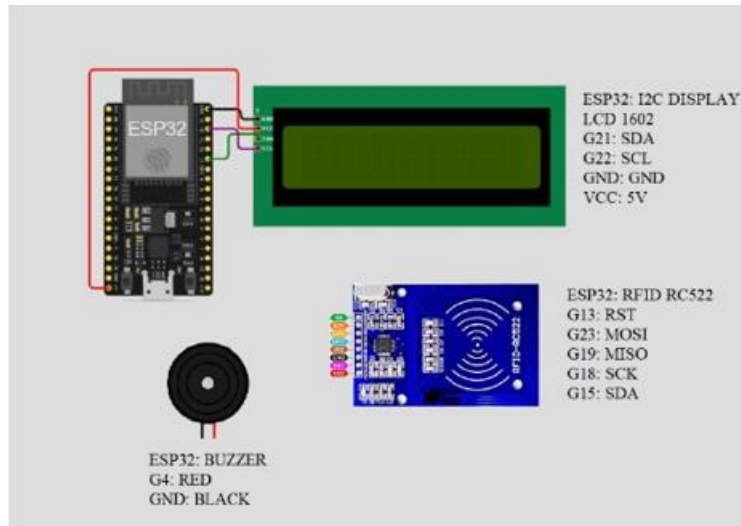
```

}
}
}

```

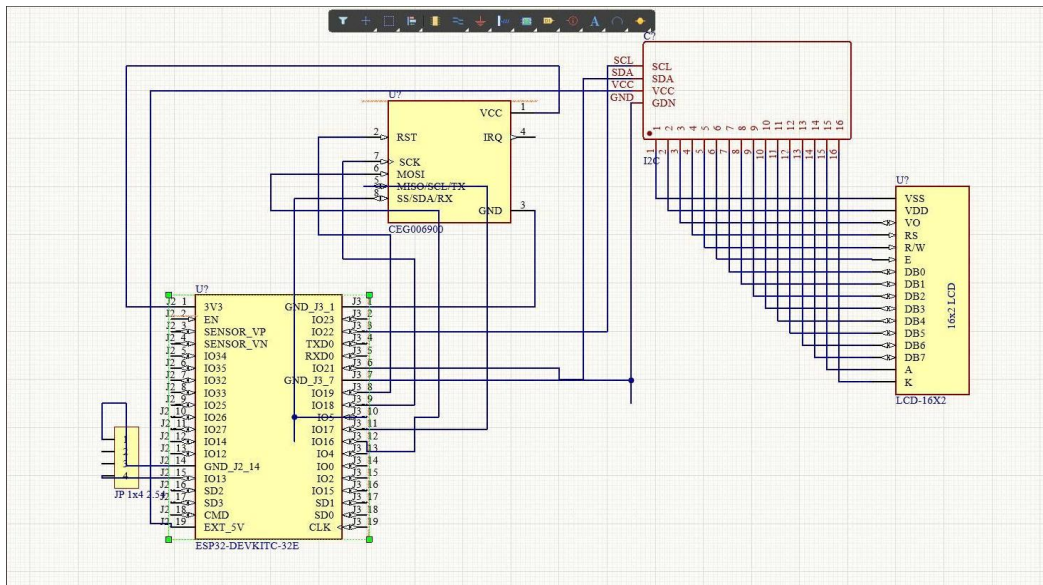
3.2. Sản phẩm hoàn thiện:

3.2.1. Sơ đồ kết nối chân:



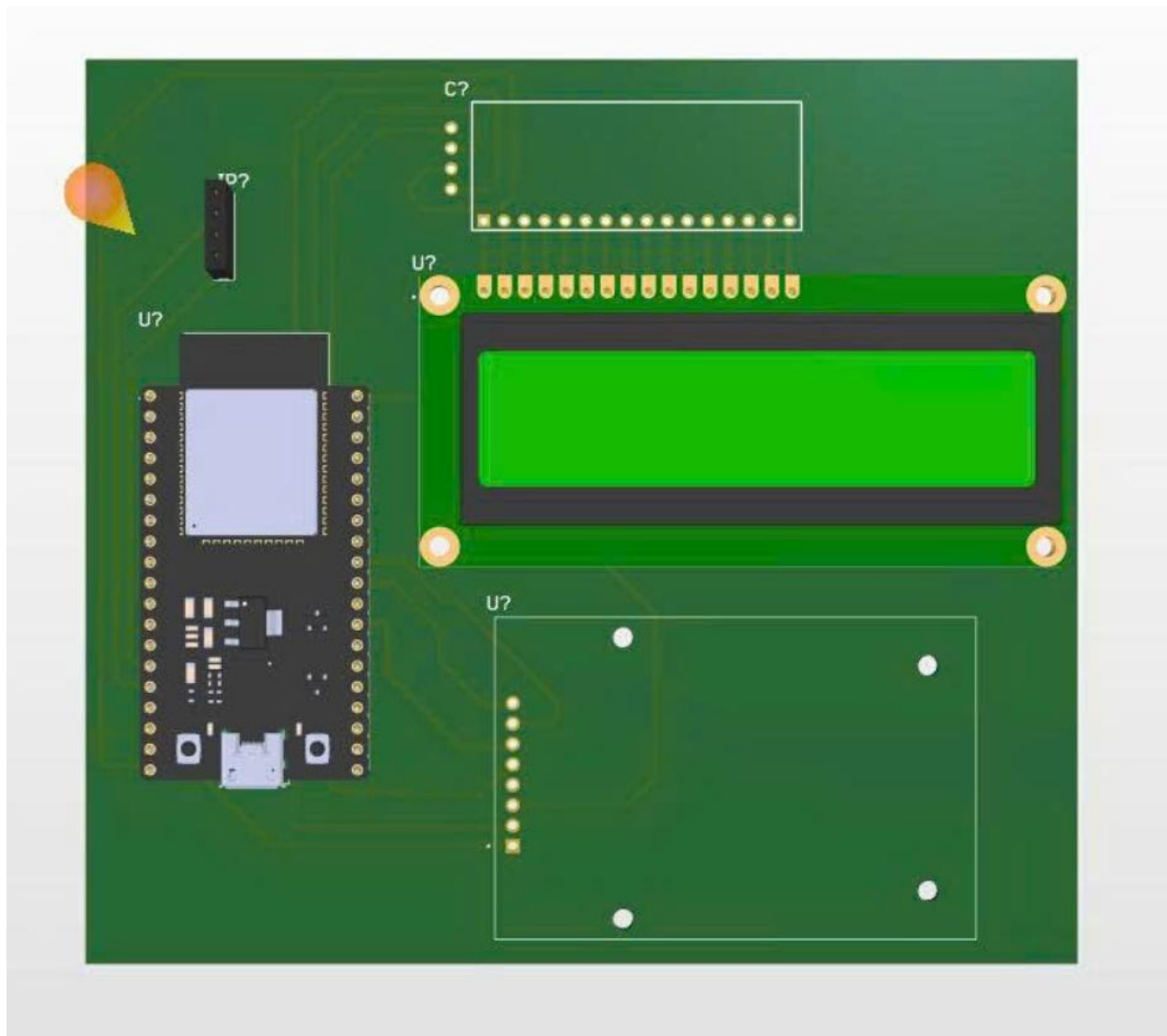
Hình 3.3: Sơ đồ kết nối chân hoàn thiện

3.2.2. Sơ đồ Schematic:



Hình 3.4: Sơ đồ Schematic

3.2.3. Sơ đồ mạch in 3D:



Hình 3.5: Sơ đồ mạch in 3D

3.2.4. Sản phẩm thực tế sau hoàn thiện:



Hình 3.6: Mặt trên sản phẩm thực tế sau hoàn thiện



Hình 3. 7: Bên trong hộp của sản phẩm thực tế sau hoàn thiện

3.2.5. Source Code:

```
#include <SPI.h>
#include <MFRC522.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <WiFi.h>
#include <HTTPClient.h>

// --- CẤU HÌNH CHÂN SPI CHO ĐẦU ĐỌC THẺ ---
#define SCK_PIN 18
#define MISO_PIN 17
#define MOSI_PIN 16
#define SS_PIN 5
#define RST_PIN 19

// --- CẤU HÌNH CHÂN I2C CHO MÀN HÌNH LCD ---
#define I2C_SDA 21
#define I2C_SCL 22

// --- CẤU HÌNH CHÂN CHO CÒI BUZZER ---
#define BUZZER_PIN 13 // Đã chuyển sang chân I013 theo yêu cầu của bạn

MFRC522 mfrc522(SS_PIN, RST_PIN);
LiquidCrystal_I2C lcd(0x27, 16, 2);

const char* ssid = "iot";
const char* password = "zx123456";
const char* googleScriptUrl =
"https://script.google.com/macros/s/AKfycbw9Tqzkb29EdZId2gTXovh7YGWE3wQDm4TU1qhIY2410okf4UxTKw1sTudIXU4CdZFz/exec";

void setup() {
    Serial.begin(115200);

    // Khởi tạo chân điều khiển Buzzer
    pinMode(BUZZER_PIN, OUTPUT);
    digitalWrite(BUZZER_PIN, LOW); // Đảm bảo còi tắt lúc ban đầu

    // 1. Khởi tạo Màn hình LCD trước tiên một cách tường minh
    Wire.begin(I2C_SDA, I2C_SCL);
    lcd.init();
    lcd.backlight();
    lcd.setCursor(0, 0);
```



```

lcd.print("Khoi dong...");

// 2. Khởi tạo Đầu đọc thẻ RFID
SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN, SS_PIN);
mfrc522.PCD_Init();

// 3. Kết nối WiFi
WiFi.begin(ssid, password);
Serial.print("Dang ket noi WiFi");

// Đặt giới hạn thời gian chờ WiFi (tránh bị kẹt mãi mãi)
int attempts = 0;
while (WiFi.status() != WL_CONNECTED && attempts < 30) {
    delay(500);
    Serial.print(".");
    attempts++;
}

lcd.clear();
if (WiFi.status() == WL_CONNECTED) {
    lcd.print("WiFi OK!");
    Serial.println("\nWiFi Da Ket Noi!");

    // Bíp 2 tiếng ngắn báo hiệu kết nối mạng thành công
    digitalWrite(BUZZER_PIN, HIGH); delay(100); digitalWrite(BUZZER_PIN, LOW);
    delay(100);
    digitalWrite(BUZZER_PIN, HIGH); delay(100); digitalWrite(BUZZER_PIN, LOW);
} else {
    lcd.print("Loi ket noi WiFi");
    Serial.println("\nKhong the ket noi WiFi!");
}

delay(2000);
lcd.clear();
lcd.print("He thong san sang");
lcd.setCursor(0, 1);
lcd.print("Moi quet the...");
}

void loop() {
    // Quan trọng: Giúp ESP32 có thời gian "thở", tránh lỗi Core Panic tự reset
    delay(50);

    if (!mfrc522.PICC_IsNewCardPresent()) return;
    if (!mfrc522.PICC_ReadCardSerial()) return;

    // PHÁT 1 TIẾNG BÍP NGẮN KHI PHÁT HIỆN THẺ VỪA QUỆT
    digitalWrite(BUZZER_PIN, HIGH);

```

```

delay(150);
digitalWrite(BUZZER_PIN, LOW);

// Xử lý mã UID
String uid = "";
for (byte i = 0; i < mfrc522.uid.size; i++) {
    uid += String(mfrc522.uid.uidByte[i] < 0x10 ? "0" : "");
    uid += String(mfrc522.uid.uidByte[i], HEX);
}
uid.toUpperCase();

Serial.println("The vua quet: " + uid);
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Dang gui du lieu");

if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    String url = String(googleScriptUrl) + "?uid=" + uid;

    http.begin(url);
    http.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);

    int httpCode = http.GET();
    lcd.clear();

    if (httpCode > 0) {
        String feedback = http.getString();
        Serial.println("Google tra ve: " + feedback);
        lcd.print("Ket qua:");
        lcd.setCursor(0, 1);
        lcd.print(feedback.substring(0, 16));

        // PHÁT 1 TIẾNG BÍP DÀI BÁO HIỆU ĐÃ GỬI LÊN GOOGLE SHEETS THÀNH CÔNG
        digitalWrite(BUZZER_PIN, HIGH);
        delay(500);
        digitalWrite(BUZZER_PIN, LOW);

    } else {
        Serial.println("Loi HTTP: " + String(httpCode));
        lcd.print("Loi mang!");
        lcd.setCursor(0, 1);
        lcd.print("Code: " + String(httpCode));
    }
    http.end();
} else {
    lcd.clear();
    lcd.print("Mat ket noi WiFi");
}

```

```

}

delay(3000);
lcd.clear();
lcd.print("He thong san sang");
lcd.setCursor(0, 1);
lcd.print("Moi quet the...");
}

```

CHƯƠNG 4: KẾT LUẬN

4.1. Kết quả đạt được:

4.1.1. Về mặt lý thuyết:

Thông qua quá trình nghiên cứu và thực hiện đồ án, em đã củng cố và mở rộng đáng kể kiến thức nền tảng về thiết kế hệ thống nhúng và Internet of Things (IoT):

- Hệ thống hóa tư duy thiết kế phần cứng: Hiểu rõ nguyên lý hoạt động và cách thức giao tiếp của vi điều khiển ESP32 với các module ngoại vi thông qua các chuẩn giao tiếp công nghiệp như SPI (với module RFID RC522) và I2C (với màn hình LCD).
- Nâng cao kỹ năng lập trình nhúng: Vận dụng thành thạo ngôn ngữ C/C++ để xử lý logic điều khiển, quản lý luồng dữ liệu phần cứng và tối ưu hóa mã nguồn cho vi điều khiển.
- Nắm vững giao thức mạng và điện toán đám mây: Hiểu sâu sắc về cách thức thiết lập kết nối Wi-Fi, xử lý các bản tin HTTP/HTTPS và sử dụng ngôn ngữ Google Apps Script (GAS) để tạo cầu nối giao tiếp giữa phần cứng và nền tảng Google Sheet

4.1.2. Về mặt thực nghiệm:

Đồ án đã được hiện thực hóa thành một mô hình phần cứng hoạt động ổn định với những kết quả khả quan:

- Xây dựng thành công hệ thống: Đã lắp ráp và lập trình hoàn thiện một thiết bị chấm công nhỏ gọn, kết hợp nhịp nhàng giữa đầu đọc RFID, vi điều khiển ESP32, màn hình LCD và còi Buzzer để phản hồi trực quan cho người dùng.
- Đồng bộ hóa dữ liệu thời gian thực: Hệ thống giải quyết xuất sắc bài toán truyền tải dữ liệu. Ngay khi nhân viên quẹt thẻ, thông tin định danh được xử lý và ghi nhận tự động vào bảng tính Google Sheets một cách chính xác, nhanh chóng.
- Vận hành trơn tru quy trình quản lý: Thay thế hoàn toàn thao tác điểm danh thủ công, cung cấp cho người quản lý một bảng tính trực tuyến tự động cập nhật, dễ dàng theo dõi và trích xuất báo cáo ở bất kỳ đâu có Internet.

4.2. Hạn chế còn tồn đọng:

Dù hệ thống cơ bản đã đáp ứng được các mục tiêu đề ra ban đầu, nhưng xét trên góc độ một sản phẩm thương mại, đề tài vẫn còn tồn tại một số hạn chế nhất định:

- Phụ thuộc hoàn toàn vào mạng Internet: Hệ thống hiện tại chưa có cơ chế lưu trữ cục bộ (Offline Mode). Nếu kết nối Wi-Fi bị gián đoạn tại thời điểm quẹt thẻ, dữ liệu điểm danh có thể bị mất hoặc hệ thống sẽ bị treo.
- Bảo mật thẻ vật lý chưa cao: Loại thẻ RFID tần số 13.56 MHz thông thường (như MIFARE Classic) có rủi ro bị sao chép (clone) mã UID, dẫn đến tình trạng chấm công hộ nếu không có các biện pháp xác thực chéo.
- Giới hạn của nền tảng lưu trữ: Việc sử dụng Google Sheets rất tiện lợi cho quy mô nhỏ, nhưng đối với các doanh nghiệp lớn có hàng ngàn lượt quẹt thẻ mỗi ngày, tốc độ phản hồi của Google Apps Script có thể bị chậm và dễ chạm ngưỡng giới hạn truy cập (quota) của Google.
- Thiết kế phần cứng mang tính thử nghiệm: Mô hình hiện tại vẫn đang được lắp ráp trên các board mạch thử nghiệm (breadboard/mạch đục lỗ), chưa được thiết kế mạch in (PCB) chuyên dụng và vỏ hộp công nghiệp, do đó khả năng chống nhiễu và độ bền cơ học chưa cao.

4.3. Hướng phát triển trong tương lai:

Để hoàn thiện hệ thống và đưa vào ứng dụng thực tiễn ở quy mô lớn hơn, em định hướng phát triển đề tài trong thời gian tới theo các tiêu chí sau:

- Tích hợp tính năng lưu trữ Offline: Bổ sung thêm module thẻ nhớ MicroSD hoặc sử dụng bộ nhớ Flash/EEPROM nội của ESP32 để lưu tạm dữ liệu khi mất mạng. Khi có Wi-Fi trở lại, hệ thống sẽ tự động đẩy toàn bộ dữ liệu bị trễ lên đám mây.
- Nâng cấp nền tảng quản trị dữ liệu: Chuyển đổi từ Google Sheets sang việc tự xây dựng một hệ thống máy chủ (Web Server) và cơ sở dữ liệu riêng (như MySQL, MongoDB). Đi kèm với đó là một giao diện Dashboard chuyên nghiệp để quản lý phân quyền và xuất báo cáo tự động.
- Tăng cường bảo mật: Nghiên cứu áp dụng cơ chế đọc/ghi mật khẩu vào các phân vùng bảo mật của thẻ RFID thay vì chỉ đọc mã UID cơ bản, hoặc kết hợp thêm xác thực sinh trắc học (vân tay) để chống gian lận.
- Hoàn thiện sản phẩm thương mại: Tiến hành vẽ mạch in (PCB) tích hợp toàn bộ linh kiện lên một bo mạch duy nhất và thiết kế vỏ hộp in 3D, giúp thiết bị trở nên thẩm mỹ, nhỏ gọn và hoạt động bền bỉ hơn trong môi trường doanh nghiệp.

TÀI LIỆU THAM KHẢO

- [1] Espressif Systems (2023), ESP32 Series Datasheet, Version 4.3, https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.
- [2] NXP Semiconductors (2016), MFRC522: Standard performance MIFARE and NTAG frontend, Rev. 3.9, <https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf>.
- [3] Texas Instruments (2015), PCF8574 Remote 8-Bit I/O Expander for I2C Bus, <https://www.ti.com/lit/ds/symlink/pcf8574.pdf>.
- [4] Google Developers (2026), Google Apps Script Reference: Spreadsheet Service, <https://developers.google.com/apps-script/reference/spreadsheet>.

[5] Arduino (2026), Arduino Reference, <https://www.arduino.cc/reference/en/>.